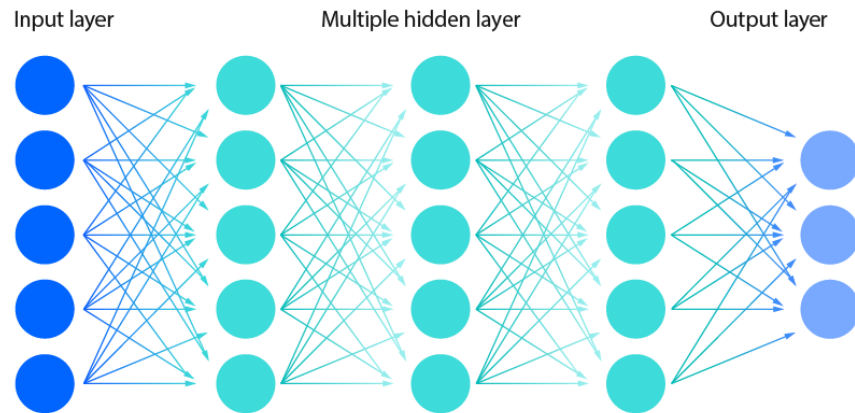


Deep neural network



NEURAL NETWORK ARCHITECTURES AND APPLICATIONS

A Clear and Practical Guide from Perceptrons to Transformers

Shaon Khan

BSc in Information and Communication Technology (ICT)
Institute of Information Technology(IIT), Jahangirnagar University

Neural Network Architectures and Applications

A Clear and Practical Guide from Perceptrons to Transformers

Shaon Khan

BSc in Information and Communication Technology (IIT)
Jahangirnagar University

2026

Neural Network Architectures and Applications

A Clear and Practical Guide from Perceptrons to Transformers

Author: Shaon Khan

Affiliation: BSc in Information and Communication Technology (IIT), Jahangirnagar University

Publication Year: 2026

Copyright © 2026 Shaon Khan. All rights reserved.

This book is prepared for educational purposes to support undergraduate study of neural networks and deep learning. No part of this material may be reproduced or distributed for commercial purposes without the written consent of the author.

Preface

This book began as a set of personal study notes on neural networks, written while trying to make sense of a field that can feel overwhelming to a newcomer. Deep learning sits at the center of some of the most talked-about technology of our time — chatbots that hold a conversation, cameras that recognize faces, and cars that drive themselves. Yet the textbooks and papers that explain how these systems work are often written for people who already know the mathematics of machine learning.

Neural networks matter because they are the engine behind almost every recent advance in artificial intelligence. A student who understands how a neuron computes a weighted sum, how a network learns from its mistakes, and how a Transformer pays attention to words in a sentence has the foundation needed to understand nearly any modern AI system, from medical image analysis to large language models.

Beginners often struggle with deep learning for three reasons. First, the mathematics is introduced too quickly, before the intuition is built. Second, technical terms are used without being clearly defined. Third, most material jumps straight to code without explaining the underlying idea in plain language. This book tries to solve all three problems by explaining every idea in simple English first, introducing formulas only after the intuition is clear, and connecting every architecture to a real, practical example.

By the end of this book, the reader will understand the full landscape of neural network architectures — from the single artificial neuron to Convolutional Neural Networks, Recurrent Neural Networks, LSTMs, Autoencoders, Generative Adversarial Networks, and Transformers — along with how these architectures power real applications such as medical imaging, speech recognition, machine translation, self-driving cars, and generative AI.

This book is written for undergraduate students of Computer Science, Information Technology, and related fields who are encountering deep learning for the first time, as well as for any self-directed learner who wants a clear and honest introduction to the subject without being buried in unnecessary jargon.

Shaon Khan

Jahangirnagar University, 2026

How to Read This Book

This book is designed to be read in order. Each chapter builds directly on the ideas introduced in the chapter before it. Readers who are completely new to neural networks should follow the sequence below rather than jumping directly to a specific architecture.

1. Learn the basic neural network idea: neurons, weights, bias, and layers (Chapter 1).
2. Understand forward propagation: how information flows from input to output (Chapter 1).
3. Learn activation functions and why non-linearity is necessary (Chapter 1).
4. Understand loss functions and backpropagation: how a network learns (Chapter 1).
5. Learn Convolutional Neural Networks (CNNs) for image data (Chapter 1).
6. Learn Recurrent Neural Networks (RNNs) and LSTMs for sequence data (Chapter 2).
7. Learn Autoencoders for compression and reconstruction (Chapter 2).
8. Learn Generative Adversarial Networks (GANs) and Transformers (Chapter 3).
9. Study the historical path that produced today's deep learning field (Chapter 4).
10. Explore real-world applications across vision, speech, and autonomous systems (Chapter 5).
11. Move on to advanced and frontier topics such as large language models, efficiency, and AI safety (Chapter 6).

Each chapter ends with a Chapter Summary, a list of Key Terms, and Review Questions. Use the Review Questions to check understanding before moving to the next chapter. Callout boxes marked KEY CONCEPT, IMPORTANT, REAL-WORLD EXAMPLE, REMEMBER, COMMON MISTAKE, NOTE, and WARNING highlight the ideas that deserve extra attention.

Table of Contents

Contents

Preface	iii
How to Read This Book	iv
Table of Contents	v
1.1 What Is Artificial Intelligence?	1
1.2 What Is Machine Learning?	1
1.3 What Is Deep Learning?	1
1.4 What Is an Artificial Neural Network?	1
1.5 Why Neural Networks Are Useful	2
1.6 Basic Structure of a Neural Network	2
1.7 How a Single Neuron Works	3
1.8 Forward Propagation	4
1.9 Activation Functions	4
Step Function.....	4
Sigmoid	4
Tanh.....	5
ReLU (Rectified Linear Unit)	5
Leaky ReLU	5
Softmax.....	5
1.10 Loss Functions	6
1.11 How Neural Networks Learn.....	6
1.12 Backpropagation	6
1.13 Classification vs Regression	7
1.14 Choosing the Output Activation	8
1.15 Artificial Neural Network Limitations	8
1.16 Introduction to CNN.....	8
1.17 How CNN Learns Visual Features	9
1.18 CNN Architecture.....	9
1.19 CNN Applications	10

1.20 ANN vs CNN	10
Chapter Summary	10
Key Terms.....	11
Review Questions	11
2.1 What Is Sequential Data?	12
2.2 Why Sequence Order Matters	12
2.3 Recurrent Neural Networks	12
2.4 RNN Architecture	12
2.5 Hidden State	12
2.6 How RNN Processes a Sequence	13
2.7 Short-Term Memory	13
2.8 Vanishing Gradient Problem.....	13
2.9 Exploding Gradient Problem.....	13
2.10 Long Short-Term Memory Networks	13
2.11 LSTM Memory Cell.....	14
2.12 RNN vs LSTM	14
2.13 Applications of RNN and LSTM.....	15
2.14 Autoencoders	15
2.15 Encoder	16
2.16 Bottleneck.....	16
2.17 Embedding	16
2.18 Decoder	16
2.19 Reconstruction	16
2.20 Denoising Autoencoders	16
2.21 Applications of Autoencoders	17
2.22 RNN vs LSTM vs Autoencoder	17
Chapter Summary	17
Key Terms.....	17
Review Questions	17
3.1 What Is Generative AI?	19
3.2 Generative vs Discriminative Models	19
3.3 Generative Adversarial Networks	19
3.4 Generator	19
3.5 Discriminator	19

3.6 GAN Training Process.....	19
3.7 GAN Applications.....	20
3.8 Limitations of GANs	20
3.9 Introduction to Attention	20
3.10 Why RNNs Have Limitations	21
3.11 Transformer Architecture	21
3.12 Self-Attention	21
3.13 Query, Key and Value	21
3.14 Positional Encoding	22
3.15 Encoder and Decoder	22
3.16 Parallel Processing	22
3.17 Transformer vs RNN/LSTM	22
3.18 Large Language Models.....	23
3.19 Transformers and Generative AI	23
3.20 Real-World Applications.....	23
Chapter Summary	23
Key Terms.....	24
Review Questions	24
4.1 Early Artificial Neurons	25
4.2 Rosenblatt Perceptron – 1958	25
4.3 Perceptron Limitations.....	25
4.4 XOR Problem	25
4.5 First AI Winter	25
4.6 Backpropagation – 1986	26
4.7 LeNet-5 – 1998.....	26
4.8 ImageNet.....	26
4.9 AlexNet – 2012	26
4.10 GPUs and Deep Learning	26
4.11 ReLU and Faster Training.....	27
4.12 AlphaGo – 2016	27
4.13 Transformer – 2017	27
4.14 Modern Large Language Models	27
4.15 Generative AI Era.....	27
Chapter Summary	28

Key Terms.....	29
Review Questions	29
Part A: Computer Vision	30
5.1 Image Classification	30
5.2 Super-Resolution.....	30
5.3 Image Colorization.....	30
5.4 Object Detection	31
5.5 YOLO	31
5.6 Medical Imaging	31
5.7 Security and Surveillance	31
5.8 Manufacturing Quality Control	32
Part B: Sequential and Speech Processing	32
5.9 Machine Translation	32
5.10 Text Summarization	32
5.11 Speech Recognition	32
5.12 Whisper	33
5.13 Text-to-Speech	33
5.14 ECG Signal Processing	33
5.15 ECG Denoising	33
5.16 Heartbeat and Arrhythmia Analysis.....	33
5.17 CNN + LSTM / Transformer for Signal Processing	34
Part C: Autonomous Systems and Generative Media.....	34
5.18 Self-Driving Systems.....	34
5.19 Deep Reinforcement Learning	35
5.20 Game-Playing AI	35
5.21 AlphaGo and AlphaZero	35
5.22 Diffusion Models	35
5.23 Text-to-Image Generation	36
5.24 Multimodal Generative AI	36
5.25 Code Generation	36
5.26 Creative Applications	36
Chapter Summary	37
Key Terms.....	37
Review Questions	37

6.1 Foundation Models.....	38
6.2 Large Language Models	38
6.3 Pretraining	38
6.4 Fine-Tuning	38
6.5 Scaling Laws	38
6.6 Parameters	38
6.7 Prompt Engineering	39
6.8 RLHF	39
6.9 Multimodal AI	39
6.10 Unified Representations / Embeddings	40
6.11 Edge AI.....	40
6.12 Why Large Models Are Difficult to Run on Small Devices	40
6.13 Quantization	40
6.14 Pruning	41
6.15 Knowledge Distillation	41
6.16 Model Efficiency	41
6.17 Interpretability	41
6.18 Explainable AI	42
6.19 Mechanistic Interpretability	42
6.20 AI Hallucination	42
6.21 Hallucination Reduction	42
6.22 AI Safety.....	42
6.23 AI Alignment	42
6.24 Responsible AI	42
6.25 Future Direction of Neural Networks	42
Chapter Summary	43
Key Terms.....	43
Review Questions	43
Book Summary	44
Architecture Comparison Matrix.....	45
Application Matrix	46
Glossary	47

CHAPTER 1

Core Foundations of Neural Networks

This chapter builds the foundation for everything else in this book. It starts with the basic ideas behind artificial intelligence, machine learning, and deep learning, then moves step by step into how a single neuron computes a value, how a full network makes a prediction, and how that network learns from its mistakes. By the end of this chapter, the reader will also understand Convolutional Neural Networks, the architecture that gave machines the ability to understand images.

1.1 What Is Artificial Intelligence?

Artificial Intelligence, or AI, is the broad field of building machines that can perform tasks that normally require human intelligence. This includes recognizing images, understanding language, playing games, and making decisions. AI is the largest umbrella term in this book — it contains many different approaches, and machine learning is simply the most successful approach used today.

1.2 What Is Machine Learning?

Machine Learning (ML) is a branch of AI in which a computer learns patterns from data instead of being told exact rules by a programmer. For example, instead of writing thousands of rules to detect spam email, a machine learning system studies thousands of examples of spam and normal email, and learns on its own which patterns separate the two.

KEY CONCEPT

The model learns patterns from data. The programmer does not write the rules by hand — the data teaches the model what the rules should be.

1.3 What Is Deep Learning?

Deep Learning is a specific type of machine learning that uses artificial neural networks with many layers — hence the word “deep.” Deep learning models can automatically discover useful patterns in raw data such as images, audio, and text, without a human manually selecting which features matter. This ability to learn features automatically is the main reason deep learning has become so powerful over the last decade.

1.4 What Is an Artificial Neural Network?

An Artificial Neural Network (ANN) is the basic building block of deep learning. It is loosely inspired by how neurons in the human brain pass signals to one another, although the similarity is more of an analogy than an exact copy. An ANN is a mathematical system made of layers of small computing units called neurons, connected to each other so information can flow from input to output.

1.5 Why Neural Networks Are Useful

Neural networks are useful because they can approximate very complex relationships between inputs and outputs — relationships that are too complicated to describe with a simple formula. Given enough data and enough training, a neural network can learn to recognize handwriting, translate languages, or predict prices, tasks that are extremely difficult to solve with traditional hand-written rules.

1.6 Basic Structure of a Neural Network

Every neural network is organized into layers of neurons. Data enters through the input layer, passes through one or more hidden layers, and produces a result at the output layer. Because every neuron in one layer is typically connected to every neuron in the next layer, this basic design is called a Dense or Fully-Connected Network.

- **Input layer:** receives the raw numbers that describe the problem, such as square footage or pixel brightness.
- **Hidden layer:** performs calculations on the data and passes the result forward. A network can have one or many hidden layers.
- **Output layer:** produces the final prediction, such as a price, a category, or a probability.
- **Neuron:** a single computing unit that takes inputs, combines them, and produces one output value.
- **Weight:** a number that represents how important a particular input is to a particular neuron.
- **Bias:** an extra adjustable number added to a neuron's calculation, allowing the network more flexibility.
- **Weighted sum:** the result of multiplying each input by its weight and adding everything together, including the bias.
- **Activation function:** a rule applied to the weighted sum that decides how strongly the neuron's signal should pass forward.

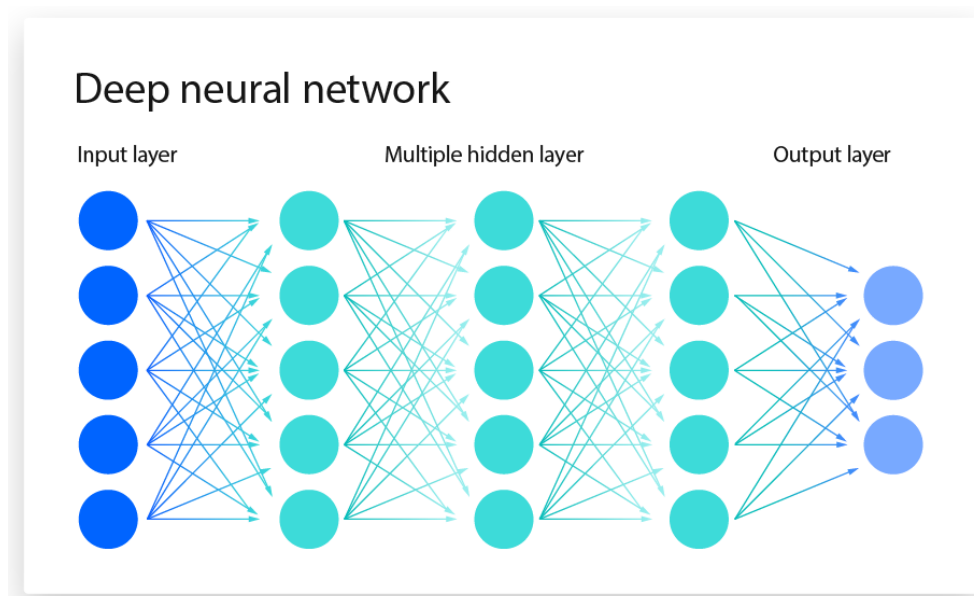


Figure 1.1. Basic Structure of an Artificial Neural Network

1.7 How a Single Neuron Works

A single neuron performs one simple calculation: it multiplies each input by its matching weight, adds all these products together, and then adds a bias term. Mathematically, this is written as:

$$z = \mathbf{w}^T \mathbf{x} + b$$

In plain English: z is the neuron's raw output before activation. w is the list of weights, one for each input. x is the list of input values. The symbol $w^T x$ means “multiply each weight by its matching input, then add all the results together.” Finally, b is the bias, a single number added at the end. Once z is calculated, it is passed through an activation function to produce the neuron's final output.

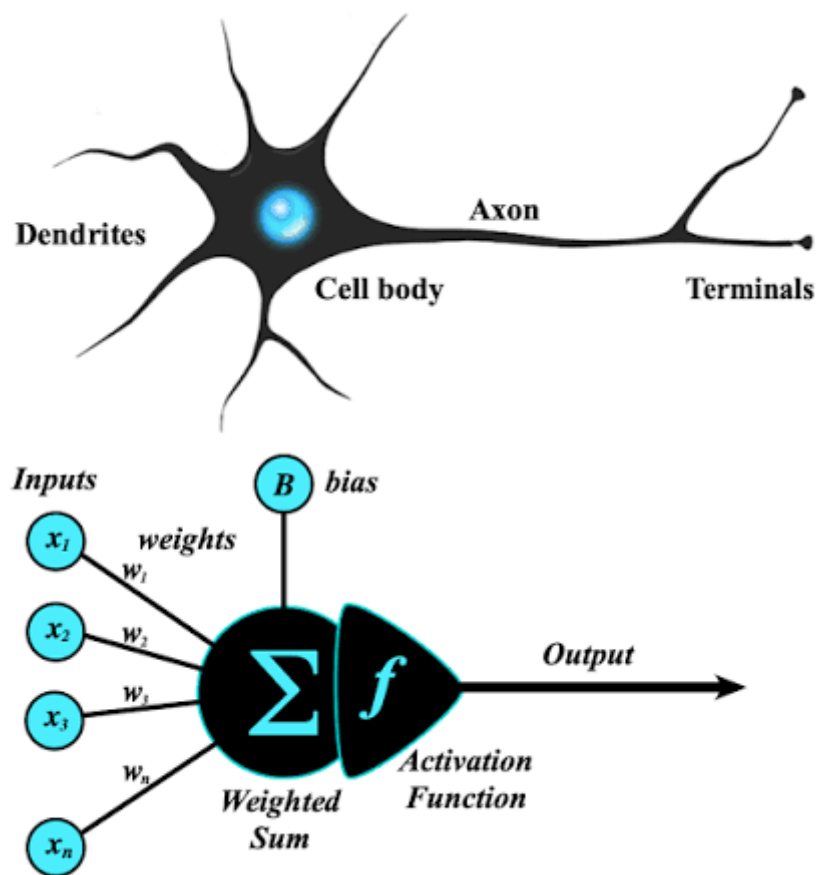


Figure 1.2. How a Single Neuron Computes Its Output

REAL-WORLD EXAMPLE

Predicting a house price from square footage and number of bedrooms is like a neuron: square footage might get a weight of 150 and bedrooms a weight of 8000. The neuron multiplies each value by its weight, adds them together with a bias, and produces a price.

1.8 Forward Propagation

Forward propagation is the process of pushing data through the entire network, from the input layer to the output layer, one layer at a time. At each layer, the network computes a weighted sum, adds a bias, and applies an activation function, then passes the result to the next layer.



Figure 1.3. The Forward Propagation Process

This process repeats layer after layer until the final output layer produces the network's prediction. Forward propagation is how a trained network makes a prediction on new data; it is also the first step during training, before the network checks how wrong its prediction was.

1.9 Activation Functions

An activation function introduces non-linearity into a neural network. Without activation functions, stacking many layers would be mathematically equivalent to a single layer, because a chain of purely linear calculations is still linear. Non-linearity is what allows a network to learn complex, curved patterns instead of only straight-line relationships.

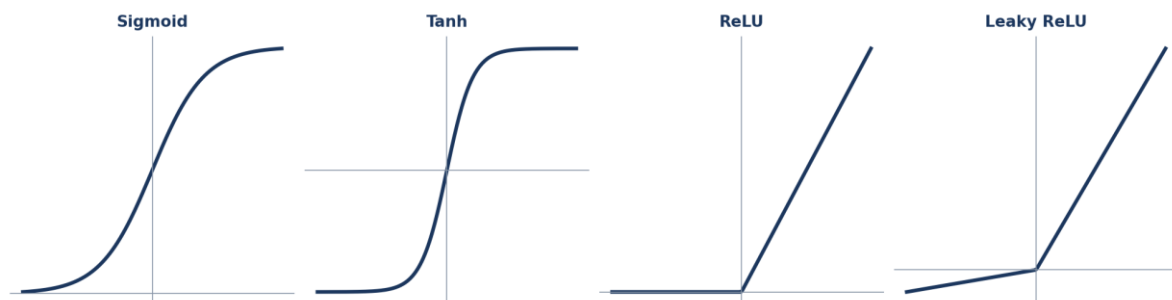


Figure 1.4. Comparison of Common Activation Functions

Step Function

Definition: outputs exactly 0 or exactly 1, depending on whether the input is below or above a threshold. Output range: $\{0, 1\}$. Advantage: extremely simple and easy to understand. Limitation: it has no smooth gradient, so it cannot be used with gradient-based learning methods. Common use: mainly of historical interest, used in the earliest perceptron models.

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Formula: $\text{sigmoid}(z) = 1 / (1 + e^{-z})$. Definition: squashes any input into a value between 0 and 1, which makes it useful for representing probabilities. Output range: (0, 1). Advantage: smooth and easy to interpret as a probability. Limitation: for very large or very small inputs, the gradient becomes extremely small, which slows down learning — this is called the vanishing gradient problem. Common use: output layer of binary classification models.

Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Formula: $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$. Definition: similar to sigmoid but centered around zero. Output range: (-1, 1). Advantage: because its output is centered at zero, it often helps hidden layers learn faster than sigmoid. Limitation: still suffers from vanishing gradients for large inputs. Common use: hidden layers, especially in older RNN architectures.

ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

Or,

$$\text{ReLU}(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

Formula: $\text{ReLU}(z) = \max(0, z)$. Definition: outputs the input directly if it is positive, and outputs zero otherwise. Output range: $[0, \infty)$. Advantage: very fast to compute and does not suffer from vanishing gradients for positive inputs, which made training deep networks practical. Limitation: neurons can sometimes get stuck outputting zero permanently, a problem known as the “dying ReLU” issue. Common use: the default choice for hidden layers in most modern networks, including CNNs.

Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} \alpha x, & x < 0 \\ x, & x \geq 0 \end{cases}$$

Formula: $\text{LeakyReLU}(z) = z$ if $z > 0$, otherwise $0.01z$. Definition: a small modification of ReLU that allows a small, non-zero gradient when the input is negative. Output range: $(-\infty, \infty)$. Advantage: helps prevent neurons from dying completely. Limitation: the extra parameter (the leak rate) must be chosen carefully. Common use: hidden layers where dying ReLU is a known problem.

Softmax

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Formula: $\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$. Definition: converts a list of raw scores into a probability distribution, where every value is between 0 and 1 and all values add up to 1. Output range: (0, 1), summing to 1 across all classes. Advantage: gives a clean probability for each possible category. Limitation: only meaningful when used across a full set of mutually exclusive classes. Common use: output layer of multi-class classification models.

1.10 Loss Functions

A loss function measures how far a network's prediction is from the correct answer. A smaller loss means a better prediction. The choice of loss function depends on the type of task the network is solving.

$$\text{Mean Squared Error: } L = (1/n) \sum (y_i - \hat{y}_i)^2$$

Mean Squared Error (MSE) is used for regression tasks, where the output is a continuous number. It measures the average of the squared difference between the true value y_i and the predicted value $\hat{y}_i$. Squaring the difference punishes large errors more heavily than small ones.

$$\text{Binary Cross-Entropy: } L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Binary Cross-Entropy is used for binary classification tasks, where the output is one of two categories (for example, spam or not spam). It compares the predicted probability $\hat{y}$ with the actual label y , and penalizes the model heavily when it is confidently wrong.

Categorical Cross-Entropy extends the same idea to more than two categories. It compares a predicted probability distribution, typically produced by a softmax output layer, against the true category, and penalizes the model based on how much probability it assigned to the wrong classes.

1.11 How Neural Networks Learn

Learning in a neural network follows a repeating cycle: the network makes a prediction, measures how wrong that prediction was using a loss function, calculates how much each weight contributed to that error using backpropagation, and then updates every weight by a small amount to reduce the error. This cycle repeats thousands or millions of times across the training data until the network's predictions become accurate.

1.12 Backpropagation

Backpropagation is the algorithm that allows a neural network to learn from its mistakes. Think of a network as a student taking a practice test. The student first attempts every question (forward propagation), then compares their answers with the answer key and finds out how many points were lost (the error). Backpropagation is the process of tracing that error backward, question by question, to figure out exactly which part of the student's reasoning caused the mistake — and adjusting that reasoning so the same mistake is less likely next time.

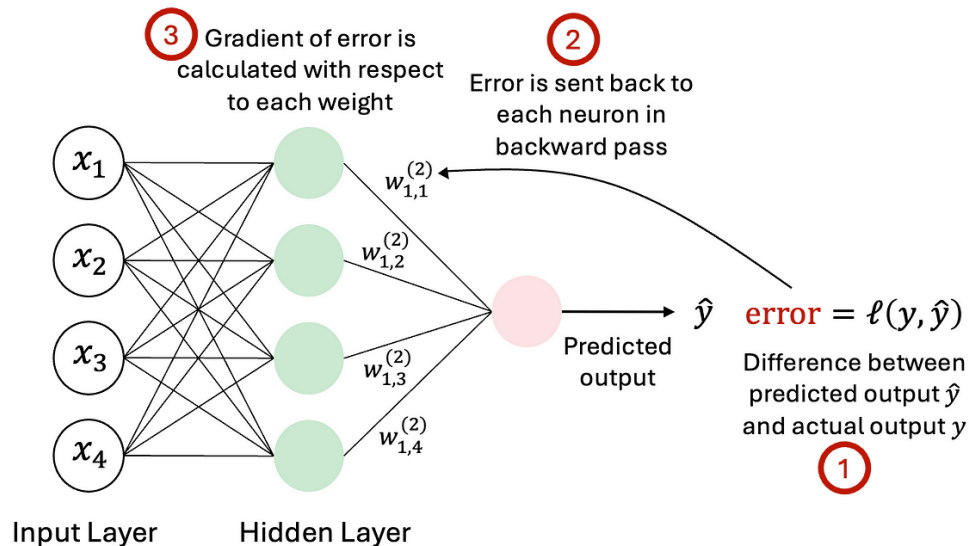


Figure 1.5. Forward Pass and Backward Pass (Backpropagation)

Backpropagation calculates the gradient of the loss with respect to each weight — in other words, it finds out how much a small change in a specific weight would change the total error. It does this efficiently by applying the chain rule of calculus, working backward from the output layer to the input layer, one layer at a time. It is important to understand that backpropagation only calculates the gradients; a separate step called the optimizer actually uses those gradients to update the weights.

$$\mathbf{w} := \mathbf{w} - \alpha (\partial L / \partial \mathbf{w})$$

This is the gradient descent update rule. $\mathbf{w}$ is the current weight, and the symbol $:=$ means “is updated to.” α (alpha) is the learning rate, a small number that controls how big each update step is. $\partial L / \partial \mathbf{w}$ is the gradient — how much the loss L would change if the weight $\mathbf{w}$ changed slightly. The formula says: move the weight a small step in the direction that reduces the loss.

COMMON MISTAKE

Backpropagation and gradient descent are not the same thing. Backpropagation calculates the gradients. Gradient descent — performed by the optimizer — uses those gradients to actually update the weights.

REMEMBER

The learning rate controls the size of each learning step. If it is too large, the network may overshoot the best solution. If it is too small, training becomes very slow.

1.13 Classification vs Regression

Regression tasks predict a continuous number, such as a house price or a temperature. Classification tasks predict a category from a fixed set of choices, such as whether an email is spam, or which digit is shown in an image. The type of task determines which loss function and which output activation function should be used.

1.14 Choosing the Output Activation

Table 1.1. Choosing the Correct Output Activation for Each Task

Task	Output	Common Activation	Example
Regression	A continuous number	None (Linear)	Predicting a house price
Binary Classification	Yes / No decision	Sigmoid	Is this email spam?
Multi-Class Classification	One choice from many	Softmax	Is this a cat, dog, or bird?
Multi-Label Classification	Several labels at once	Sigmoid (per label)	Which tags apply to this photo?

1.15 Artificial Neural Network Limitations

A standard, fully-connected ANN treats every input independently and has no built-in sense of structure. This creates two major limitations. First, images contain enormous numbers of pixels, and connecting every pixel to every neuron in the next layer would require an impractical number of weights. Second, a plain ANN has no memory — it cannot naturally handle sequences of data such as sentences or time-series signals, where the order of the data matters. These two limitations motivated the development of CNNs (Section 1.16) and RNNs (Chapter 2).

1.16 Introduction to CNN

A Convolutional Neural Network (CNN) is a neural network architecture designed specifically for visual data such as images and video. Standard ANNs struggle with images because even a small picture can contain millions of pixel values, and connecting every pixel to every neuron would create far too many weights to train efficiently. A CNN solves this by scanning small regions of the image at a time, similar to sliding a magnifying glass across a picture.

- **Pixels:** the individual colored dots that make up a digital image.
- **Convolution:** the mathematical operation of sliding a small filter across the image and computing a weighted sum at each position.
- **Filter / kernel:** a small grid of learnable weights that detects a specific pattern, such as an edge or a color transition.
- **Feature map:** the output produced after a filter has scanned across the entire image.
- **Stride:** how many pixels the filter moves between each step.
- **Padding:** extra pixels added around the border of the image so the filter can process edge pixels properly.
- **Pooling:** a downsampling step that shrinks the feature map while keeping the most important information.
- **Flattening:** converting the final grid of features into a single long list of numbers.
- **Fully connected layer:** a standard dense layer, usually placed near the end of the CNN, that combines all extracted features to make the final prediction.

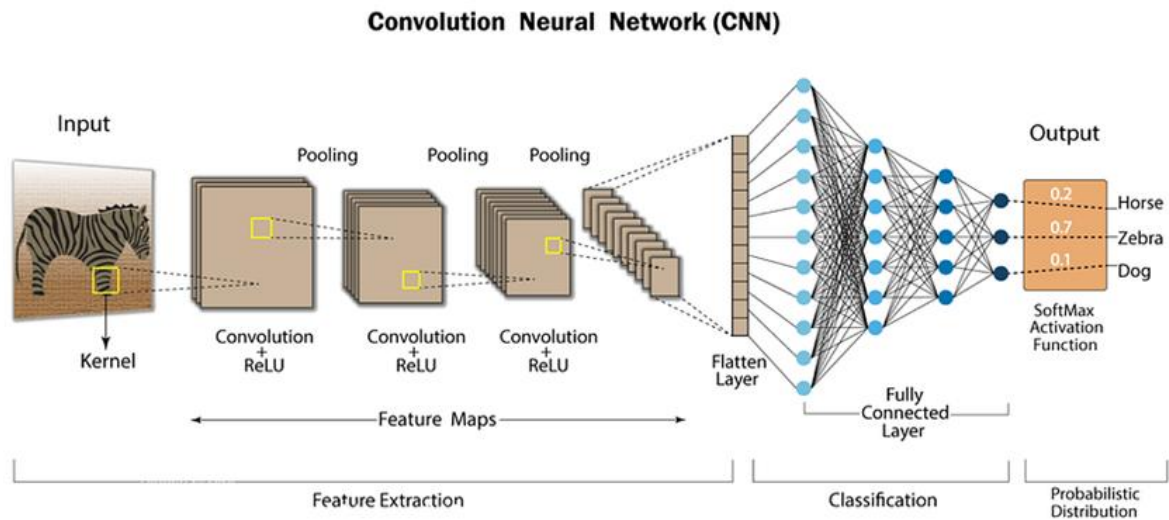


Figure 1.6. The CNN Workflow, from Image to Prediction

1.17 How CNN Learns Visual Features

CNNs learn visual features in a hierarchy. The earliest layers detect very simple patterns such as edges, corners, and color transitions. Deeper layers combine these simple patterns into more complex shapes such as curves and textures. Even deeper layers combine those shapes into recognizable parts of objects, such as eyes, wheels, or ears. The final layers combine all of this evidence to recognize a complete object.



Depth increases from left to right through the network

Figure 1.7. How CNNs Build Understanding from Edges to Objects

REAL-WORLD EXAMPLE

When a phone's photo app searches for pictures of cats, a CNN scans each photo and detects two pointed shapes, a small triangular nose, and whiskers. Combining this evidence, the network labels the photo "cat" — without ever being told by a human what a cat's ears look like.

1.18 CNN Architecture

A typical CNN architecture stacks several convolution and pooling layers, gradually reducing the spatial size of the image while increasing the number of feature maps. After several of these blocks, the

output is flattened and passed through one or more fully connected layers that combine all the learned features into a final prediction, such as a class label.

1.19 CNN Applications

- **Image classification:** assigning a single label to an entire image, such as identifying the main object shown.
- **Object detection:** locating and labeling multiple objects within a single image, covered in more depth in Chapter 5.
- **Medical imaging:** detecting tumors, fractures, or other abnormalities in X-rays, CT scans, and MRI images.
- **Face recognition:** identifying or verifying a person's identity from a photo.
- **Image restoration:** repairing blurry, noisy, or low-resolution images, discussed further in Chapter 5.

1.20 ANN vs CNN

Table 1.2. Comparing Standard ANN and CNN Architectures

Feature	ANN (Fully-Connected)	CNN
Best suited for	Tabular / numeric data	Images and visual data
Connectivity	Every neuron connects to every neuron	Small local filters scan the input
Number of weights	Very high for large inputs	Much lower, weights are shared
Understands spatial structure	No	Yes
Typical use case	House price prediction	Cat vs dog image classification

Chapter Summary

- A neural network is built from layers of neurons connected by weighted connections.
- A single neuron computes a weighted sum of its inputs, adds a bias, and applies an activation function.
- Forward propagation pushes data through the network to produce a prediction.
- Activation functions introduce non-linearity, allowing networks to learn complex patterns.
- Loss functions measure prediction error; backpropagation calculates how each weight contributed to that error.
- Gradient descent uses the gradients found by backpropagation to update the weights.
- CNNs are specialized for visual data, using convolution and pooling instead of full connectivity.

Key Terms

Activation Function • ANN • Backpropagation • Bias • CNN • Convolution • Feature Map • Filter / Kernel • Forward Propagation • Gradient • Gradient Descent • Hidden Layer • Learning Rate • Loss Function • Neuron • Padding • Pooling • ReLU • Sigmoid • Softmax • Stride • Weight

Review Questions

1. In your own words, explain what a weight and a bias represent in a single neuron.
2. Why is a non-linear activation function necessary in a neural network?
3. Describe the difference between forward propagation and backpropagation.
4. Which output activation function would you choose for a model that classifies an image as a cat, dog, or bird, and why?
5. Explain why a standard ANN is inefficient for processing large images.
6. List the steps a CNN uses to go from a raw image to a final class prediction.
7. Compare ANN and CNN in terms of how many weights they typically require for image data.

CHAPTER 2

Sequence Data and Compression

Chapter 1 introduced ANNs and CNNs, both of which treat each input independently. Many real-world problems, however, involve data where order matters — a sentence, an audio clip, or a heartbeat signal recorded over time. This chapter introduces Recurrent Neural Networks and LSTMs, which are designed for sequential data, and Autoencoders, which learn to compress and reconstruct data.

2.1 What Is Sequential Data?

Sequential data is any data where the order of the values carries meaning. A sentence is sequential because rearranging the words changes or destroys its meaning. An audio recording is sequential because sound is a signal that changes moment by moment. A patient's heartbeat signal is sequential because doctors care about how the rhythm changes over time, not just a single isolated value.

2.2 Why Sequence Order Matters

A standard ANN examines each input independently and has no memory of what came before. This is a serious limitation for sequential data. Consider the sentence “The sky is very cloudy” — predicting the next word requires remembering the earlier words in the sentence. A plain ANN, with no memory of previous inputs, cannot make this kind of prediction reliably.

2.3 Recurrent Neural Networks

A Recurrent Neural Network (RNN) is a neural network built specifically for sequential data. Unlike an ANN, an RNN contains a loop that passes information from one step in the sequence to the next. This loop gives the network a form of short-term memory, allowing it to use information from earlier in the sequence when processing the current input.

2.4 RNN Architecture

At each time step, an RNN takes two inputs: the new data point in the sequence, and the hidden state carried over from the previous time step. It combines these two inputs to produce a new hidden state, which is then passed both to the output for that time step and forward to the next time step.

2.5 Hidden State

The hidden state is the RNN's working memory. It is a set of numbers that summarizes everything the network has “seen” in the sequence so far. At every time step, this summary is updated using the new input, and the updated summary is carried forward to the next step.

2.6 How RNN Processes a Sequence

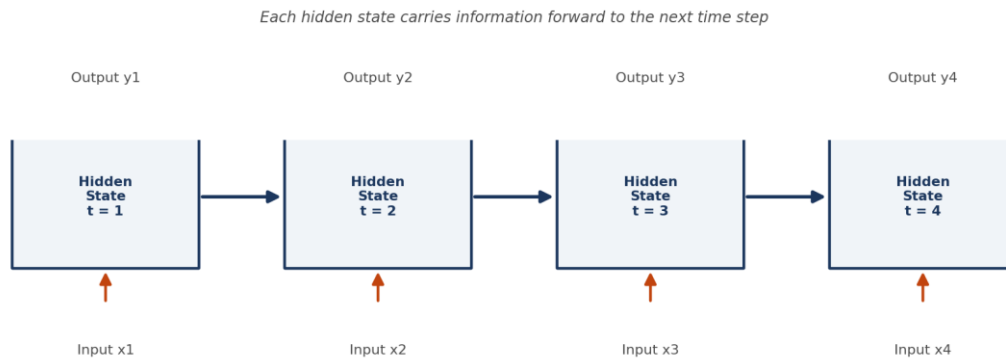


Figure 2.1. *An RNN Unrolled Across Multiple Time Steps*

Although an RNN can be drawn as a single loop, it is often easier to understand when “unrolled” across time, showing one copy of the network for each step in the sequence. Every copy shares the same weights; only the hidden state changes from step to step.

2.7 Short-Term Memory

The hidden state gives an RNN short-term memory — it can remember information from recent steps in the sequence. However, this memory fades as the sequence gets longer, because the influence of an early input gradually shrinks as more steps are processed. This limitation leads directly to the vanishing gradient problem, described next.

2.8 Vanishing Gradient Problem

When a standard RNN processes a very long sequence — such as a two-hundred-page book or a long medical signal — the gradient used to update earlier weights must travel back through many time steps during backpropagation. At each step, the gradient tends to shrink. After enough steps, the gradient becomes so small that the earliest parts of the sequence effectively stop influencing learning. This is called the vanishing gradient problem, and it means a standard RNN gradually “forgets” information from far in the past.

2.9 Exploding Gradient Problem

The opposite problem can also occur. In some cases, gradients grow larger and larger as they travel backward through time, eventually becoming so large that the weight updates become unstable and the network fails to learn. This is called the exploding gradient problem, and it is usually addressed with a technique called gradient clipping, which caps the maximum size of a gradient update.

2.10 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks were designed specifically to solve the vanishing gradient problem. An LSTM is a type of recurrent neural network that adds a special memory structure, allowing

it to preserve important information across long sequences while still being able to discard information that is no longer useful.

2.11 LSTM Memory Cell

The heart of an LSTM is the memory cell, a dedicated pathway that carries information across time with minimal interference. Three gates control what happens to this memory at every time step.

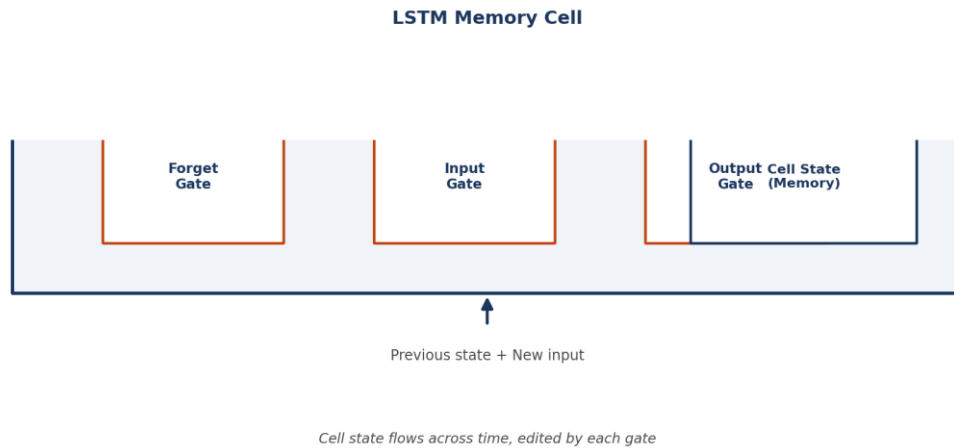


Figure 2.2. *The LSTM Memory Cell and Its Three Gates*

- **Forget Gate:** decides which parts of the old memory are no longer useful and should be discarded.
- **Input Gate:** decides which parts of the new information are useful enough to be added to memory.
- **Output Gate:** decides which parts of the memory should be passed forward as the output for this time step.

KEY CONCEPT

Think of the LSTM memory cell like a notebook. The forget gate erases old notes that are no longer needed. The input gate writes new important notes. The output gate decides what part of the notebook to read out loud right now.

2.12 RNN vs LSTM

Table 2.1. *Comparing RNN and LSTM*

Feature	Standard RNN	LSTM
Memory type	Short-term only	Short-term and long-term
Handles long sequences	Poorly, due to vanishing gradients	Well, using gated memory cells

Internal structure	Single simple loop	Loop plus forget, input, and output gates
Computation cost	Lower	Higher, due to extra gates
Typical use case	Very short sequences	Long text, audio, or signal data

2.13 Applications of RNN and LSTM

- **Text prediction:** suggesting the next word while typing, based on the words that came before.
- **Speech processing:** modeling how spoken sound changes over time before it is turned into text.
- **Time-series prediction:** forecasting future values, such as stock prices or weather, from historical data.
- **ECG analysis:** detecting irregular heartbeats by tracking how the electrical signal of the heart changes over time.
- **Sensor data:** monitoring readings from industrial or wearable sensors to detect unusual patterns.

2.14 Autoencoders

An Autoencoder is a neural network that learns to compress data into a smaller summary and then reconstruct the original data from that summary. Autoencoders are commonly used for representation learning and reconstruction. They are often trained without explicit labels, since the input itself serves as the target output, but this does not mean every use of an autoencoder is automatically unsupervised in every possible setting.

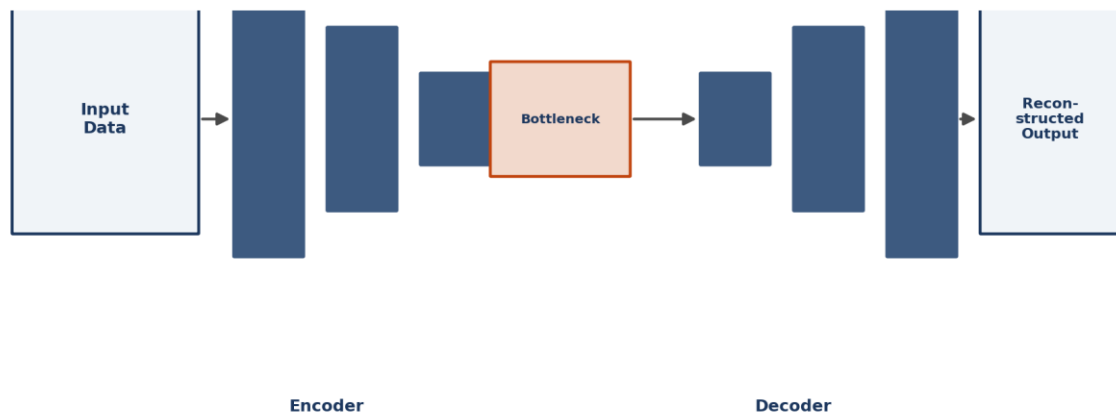


Figure 2.3. The Autoencoder: Encoder, Bottleneck, and Decoder

2.15 Encoder

The encoder is the first half of an autoencoder. It takes the original input, such as an image, and passes it through a series of layers that gradually reduce its size, forcing the network to keep only the most important information.

2.16 Bottleneck

The bottleneck, also called the embedding, is the smallest and most compressed point in the network. Because this layer is much smaller than the input, the network is forced to discard unnecessary detail and keep only the features that matter most for reconstructing the input.

2.17 Embedding

An embedding is a compact numerical representation of the input data. In an autoencoder, the bottleneck layer produces the embedding. Embeddings are useful beyond autoencoders as well — they are widely used throughout deep learning to represent words, images, or other data as lists of numbers that capture meaningful patterns.

2.18 Decoder

The decoder is the second half of an autoencoder. It takes the compressed embedding from the bottleneck and gradually expands it back into something the same size and shape as the original input, attempting to reconstruct the input as closely as possible.

2.19 Reconstruction

Reconstruction is the autoencoder's final output — its best attempt at recreating the original input from the compressed embedding. During training, the network's loss function measures the difference between the original input and the reconstruction, and the network adjusts its weights to make this difference as small as possible.

2.20 Denoising Autoencoders

A denoising autoencoder is trained on a slightly different task: it receives a noisy or corrupted version of the input, but it is still trained to reconstruct the clean, original version. Because the bottleneck can only hold the most essential information, the network learns to ignore noise and preserve the true underlying signal.

REAL-WORLD EXAMPLE

If a denoising autoencoder is given a blurry or noisy photo, the encoder captures only the essential structure of the image, discarding the noise. The decoder then reconstructs a cleaner, sharper version of the original photo.

2.21 Applications of Autoencoders

- **Image and signal denoising:** removing noise from photos or physiological signals such as ECG recordings.
- **Anomaly detection:** flagging inputs that reconstruct poorly, since unusual data often does not fit the learned patterns.
- **Dimensionality reduction:** compressing high-dimensional data into a smaller, more manageable embedding.
- **Pretraining:** learning useful data representations that can be reused for other tasks.

2.22 RNN vs LSTM vs Autoencoder

Table 2.2. Comparing RNN, LSTM, and Autoencoder

Architecture	Main Purpose	Handles Sequences?	Typical Example
RNN	Short sequence modeling	Yes, short sequences	Simple next-word prediction
LSTM	Long sequence modeling	Yes, long sequences	ECG sequence analysis
Autoencoder	Compression and reconstruction	Not required	Image denoising

Chapter Summary

- Sequential data is any data where order carries meaning, such as text, audio, or signals.
- RNNs use a loop to carry a hidden state forward, giving them short-term memory.
- Long sequences cause vanishing or exploding gradients in standard RNNs.
- LSTMs solve the vanishing gradient problem using forget, input, and output gates.
- Autoencoders compress data through an encoder into a small bottleneck, then reconstruct it with a decoder.
- Denoising autoencoders learn to recover clean data from noisy or corrupted input.

Key Terms

Autoencoder • Bottleneck • Decoder • Denoising Autoencoder • Embedding • Encoder • Exploding Gradient • Forget Gate • Hidden State • Input Gate • LSTM • Output Gate • Recurrent Neural Network • Sequential Data • Short-Term Memory • Vanishing Gradient

Review Questions

1. Explain why a standard ANN struggles with sequential data such as text or audio.
2. What is the hidden state in an RNN, and what problem does it solve?
3. Describe the vanishing gradient problem in your own words.
4. Name the three gates in an LSTM memory cell and explain what each one does.
5. Why is the bottleneck layer in an autoencoder smaller than the input layer?

6. Give one real-world example each for RNN/LSTM and for autoencoders.
7. Compare RNN, LSTM, and Autoencoder in terms of their main purpose.

CHAPTER 3

Generative and Modern Neural Network Architectures

The architectures covered so far are mostly used to analyze or classify existing data. This chapter introduces architectures that can create new data: Generative Adversarial Networks, which generate realistic images through a competition between two networks, and Transformers, the architecture behind modern large language models such as ChatGPT, Gemini, and Claude.

3.1 What Is Generative AI?

Generative AI refers to models that create new content rather than simply analyzing existing content. Instead of answering “what is in this image?” a generative model answers “create a new image that looks like this.” Generative AI can produce images, text, audio, and video that did not exist before.

3.2 Generative vs Discriminative Models

A discriminative model learns to distinguish between categories — for example, deciding whether a photo shows a cat or a dog. A generative model learns the underlying patterns of the data well enough to produce brand-new examples that resemble the training data. Many of the architectures in Chapter 1 and Chapter 2, such as CNNs used for classification, are discriminative. GANs and diffusion models, covered in this book, are generative.

3.3 Generative Adversarial Networks

A Generative Adversarial Network (GAN) is an architecture that can create entirely new data, such as realistic images of people who do not exist. A GAN's key idea is a competition between two neural networks: a Generator and a Discriminator.

3.4 Generator

The Generator starts with random noise and tries to transform it into data that looks real, such as a realistic image. Early in training, the Generator produces very poor, obviously fake results.

3.5 Discriminator

The Discriminator acts as a judge. It examines an image and predicts whether it is a real image from the training data or a fake image produced by the Generator.

3.6 GAN Training Process

REAL-WORLD EXAMPLE

A GAN works like an art forger and a detective. The Generator is the forger, trying to paint a convincing fake. The Discriminator is the detective, trying to catch the fake. As training continues, the forger gets better at fooling the detective, and the detective gets better at spotting fakes, pushing both networks to improve together.

At the start of training, the Generator's images are easy for the Discriminator to identify as fake. But as training progresses, the Generator learns from the Discriminator's feedback and produces increasingly convincing images. Eventually, the Generator can produce images so realistic that the Discriminator can no longer reliably tell real from fake.

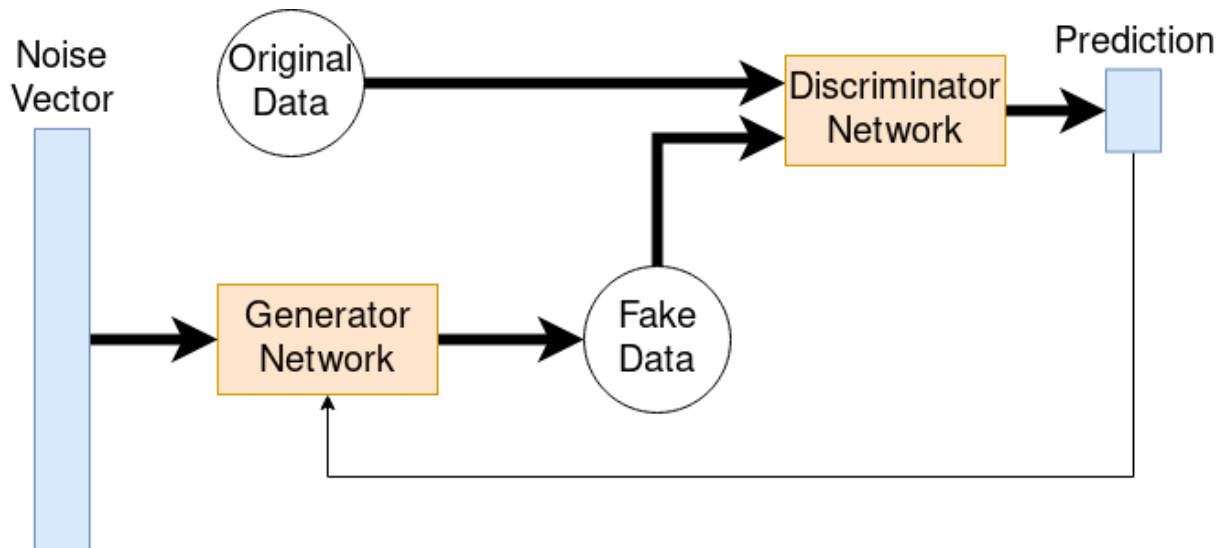


Figure 3.1. How a GAN Trains: Generator vs Discriminator

3.7 GAN Applications

- **Realistic image generation:** creating photographs of people, objects, or scenes that do not exist in reality.
- **Image and video super-resolution:** converting low-quality images into sharper, high-resolution versions, discussed further in Chapter 5.
- **Data augmentation:** generating additional realistic training examples when real data is limited.

3.8 Limitations of GANs

GANs can be difficult to train because the Generator and Discriminator must improve at a similar pace; if one becomes too strong too quickly, training can become unstable. GANs can also suffer from a problem called mode collapse, where the Generator produces only a limited variety of outputs instead of a diverse range of results.

3.9 Introduction to Attention

Attention is a mechanism that allows a neural network to focus more strongly on the parts of the input that are most relevant to the current task, rather than treating every part of the input equally. Attention is the key idea behind the Transformer architecture.

3.10 Why RNNs Have Limitations

RNNs and LSTMs process sequences one step at a time, which makes them slow to train on large datasets, since each step must wait for the previous step to finish. They can also still struggle to connect information that is very far apart in a long sequence, even with the improvements LSTMs provide. These limitations motivated the search for an architecture that could process an entire sequence at once while still understanding relationships between distant words.

3.11 Transformer Architecture

The Transformer is currently the most influential neural network architecture in modern artificial intelligence. It is important to understand that a Transformer is an architecture, not simply a chatbot — chatbots such as ChatGPT are applications built using Transformer-based models. Instead of processing words one at a time like an RNN, a Transformer processes an entire sequence in parallel and uses a mechanism called self-attention to understand how every word relates to every other word.

3.12 Self-Attention

Self-attention allows a Transformer to weigh the importance of every other word in a sentence when interpreting a specific word. Consider the sentence: “The animal did not cross the road because it was tired.” The word “it” could refer to either “animal” or “road.” Self-attention calculates a stronger connection between “it” and “animal” (and “tired”), allowing the model to correctly understand that “it” refers to the animal.

The **animal** did not cross because **it** was **tired**

The word "it" pays more attention to "animal" and "tired" than to other words

Figure 3.2. Self-Attention Connecting Related Words in a Sentence

3.13 Query, Key and Value

Self-attention is computed using three vectors derived from each word: a Query, a Key, and a Value. A helpful analogy is a library search. The Query is like a search request — what you are looking for. The Key is like the label on each book describing what it contains. The Value is the actual content of the book. The model compares the Query for one word against the Keys of every other word to decide how much attention to pay to each one, then combines the Values accordingly.

3.14 Positional Encoding

Because a Transformer processes all words at once rather than in order, it has no built-in sense of word position. Positional encoding solves this by adding information about each word's position in the sentence to its representation, so the model can still tell the difference between “the dog bit the man” and “the man bit the dog.”

3.15 Encoder and Decoder

Many Transformer models are organized into an encoder and a decoder. The encoder reads the entire input sequence and builds a rich contextual representation of it using self-attention. The decoder generates the output sequence one piece at a time, using both its own self-attention and attention over the encoder's output to stay grounded in the source information.

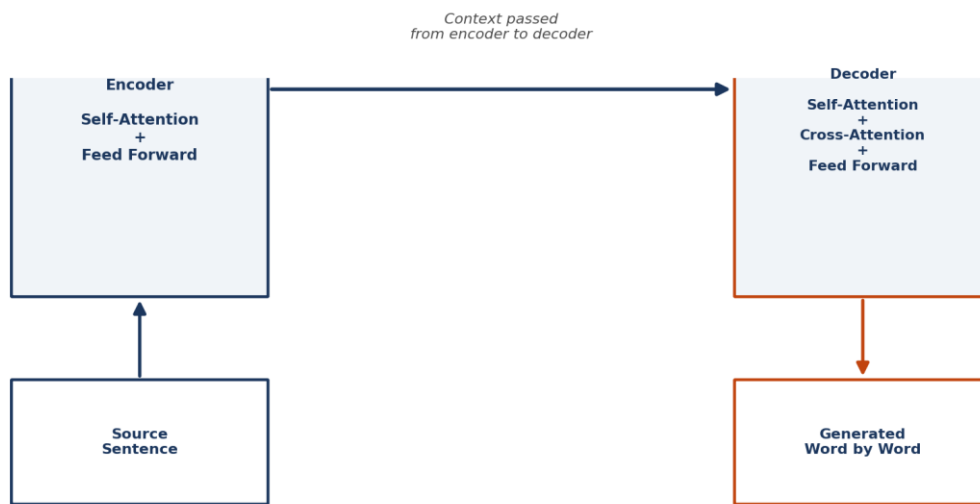


Figure 3.3. The Transformer Encoder-Decoder Structure

3.16 Parallel Processing

Unlike RNNs, which must process a sequence step by step, a Transformer processes every position in the sequence simultaneously. This parallel processing allows Transformers to be trained far more efficiently on modern hardware such as GPUs, which is one of the key reasons Transformers scaled successfully to enormous datasets.

3.17 Transformer vs RNN/LSTM

Table 3.1. Comparing Transformer and RNN/LSTM Architectures

Feature	RNN / LSTM	Transformer
Processing style	Sequential, one step at a time	Parallel, entire sequence at once
Long-range understanding	Limited, even with LSTM gates	Strong, via self-attention

Training speed on large data	Slower	Much faster
Position awareness	Built in, from sequential order	Added explicitly via positional encoding
Typical modern use	Small or legacy sequence tasks	Large language models, translation

3.18 Large Language Models

A Large Language Model (LLM) is a Transformer-based model trained on massive amounts of text to predict and generate natural language. Because of their scale and training data, LLMs can perform a wide variety of language tasks, including answering questions, writing code, and holding conversations. LLMs such as ChatGPT, Gemini, and Claude are applications built on top of Transformer architectures.

3.19 Transformers and Generative AI

Transformers are not limited to text. The same self-attention mechanism has been extended to images, audio, and video, making Transformers a core building block of modern generative AI systems that can produce text, images, and other media.

3.20 Real-World Applications

- **Machine translation:** converting text from one language to another while preserving meaning and context.
- **Text generation:** producing new, coherent passages of text from a prompt.
- **Question answering:** reading a passage or dataset and answering questions about it accurately.
- **Code generation:** writing, explaining, and debugging software code.
- **Summarization:** condensing long documents into short, accurate summaries.
- **Chatbots:** holding natural, multi-turn conversations with users.

Chapter Summary

- Generative AI creates new data rather than only analyzing existing data.
- A GAN trains a Generator and a Discriminator against each other until the Generator produces realistic output.
- Self-attention lets a Transformer weigh the relationship between every pair of words in a sequence.
- Query, Key, and Value vectors are the building blocks of the self-attention calculation.
- Positional encoding gives a Transformer a sense of word order, since it processes words in parallel.
- Transformers process sequences in parallel, making them faster to train than RNNs on large datasets.
- Large Language Models are Transformer-based systems trained on massive text datasets.

Key Terms

Attention • Decoder • Discriminator • Encoder • GAN • Generator • Large Language Model • Parallel Processing • Positional Encoding • Query • Self-Attention • Transformer • Key • Value

Review Questions

1. Explain the roles of the Generator and the Discriminator in a GAN using your own analogy.
2. Why can GAN training become unstable?
3. In the self-attention example sentence, explain why “it” attends strongly to “animal”.
4. What problem does positional encoding solve in a Transformer?
5. Compare how RNNs and Transformers process a sequence.
6. Is a Transformer the same thing as a chatbot? Explain your answer.
7. List three real-world applications of Transformer-based models.

CHAPTER 4

History and Evolution of Deep Learning

Deep learning did not become powerful overnight. It is the result of decades of trial and error, periods of excitement, periods of disappointment, and a handful of breakthroughs that changed the course of the field. This chapter tells that story from the first artificial neuron to the modern generative AI era.

4.1 Early Artificial Neurons

The earliest ideas about artificial neurons date back to the 1940s, when researchers first proposed simple mathematical models of how a biological neuron might combine and threshold signals. These early models were purely theoretical and were not yet trainable from data.

4.2 Rosenblatt Perceptron – 1958

In 1958, Frank Rosenblatt created the Perceptron, widely regarded as the earliest working form of an artificial neural network. The Perceptron was a single-layer circuit designed to make simple binary decisions based on numerical inputs, taking weighted inputs, summing them, and comparing the result against a threshold to output either 0 or 1.

4.3 Perceptron Limitations

The Perceptron could only learn patterns that could be separated by a single straight line (or, more generally, a single flat decision boundary). This meant it could not solve problems where the correct decision boundary was more complex.

4.4 XOR Problem

The XOR (Exclusive OR) problem is a simple logic puzzle where the output is true only when the two inputs are different, and false when they are the same. No single straight line can separate this pattern. In 1969, Marvin Minsky and Seymour Papert published a book mathematically proving that a single-layer Perceptron could not solve XOR or other non-linear problems.

4.5 First AI Winter

Because the hardware of the time could not efficiently train networks with multiple layers, and because Minsky and Papert's proof discouraged confidence in neural networks, funding and interest in the field declined sharply. This period of reduced progress and reduced funding became known as the first "AI Winter."

4.6 Backpropagation – 1986

In 1986, Geoffrey Hinton, David Rumelhart, and Ronald Williams popularized the backpropagation algorithm. Backpropagation made it practical to train networks with multiple hidden layers by calculating the exact contribution of each layer to the overall error and adjusting the weights accordingly. This solved the fundamental limitation that had ended the single-layer Perceptron and reopened the path toward deeper networks capable of solving non-linear problems such as XOR.

4.7 LeNet-5 – 1998

In 1998, Yann LeCun introduced LeNet-5, one of the earliest practical Convolutional Neural Networks. LeNet-5 was successfully deployed commercially by banks to read handwritten digits on checks. Even so, progress in the field remained slow, largely because computers at the time were still too weak to train much larger models efficiently.

4.8 ImageNet

ImageNet is a large dataset and annual competition containing over a million labeled images spanning a thousand categories. The ImageNet Large Scale Visual Recognition Challenge became the proving ground for computer vision algorithms and, in 2012, the stage for a breakthrough that changed the direction of AI research.

4.9 AlexNet – 2012

In 2012, Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton entered the ImageNet competition with a deep CNN called AlexNet. Traditional machine learning methods at the time achieved roughly 26 percent error on the competition. AlexNet achieved roughly 15.3 percent error — a dramatic improvement that convinced the research community that deep learning had arrived.

Table 4.1. ImageNet Competition Error Rates (2012)

Approach	Error Rate
Traditional Machine Learning	~26%
AlexNet (Deep Learning)	~15.3%

AlexNet's success relied on two key shifts. First, GPU acceleration: using Graphics Processing Units instead of CPUs to parallelize the matrix calculations at the heart of neural network training, dramatically speeding up training time. Second, the ReLU activation function, $f(x) = \max(0, x)$, which trained much faster than older activation functions and helped avoid vanishing gradients.

4.10 GPUs and Deep Learning

GPUs were originally designed to render graphics for video games, a task that requires doing many simple mathematical operations in parallel. Neural network training also requires huge numbers of parallel matrix operations, which made GPUs a natural, and eventually essential, fit for deep learning.

4.11 ReLU and Faster Training

The widespread adoption of the ReLU activation function, introduced in Section 1.9, was a key ingredient in making deep networks practical to train. Because ReLU does not saturate for positive inputs the way sigmoid and tanh do, networks using ReLU could be trained faster and with more layers.

4.12 AlphaGo – 2016

In 2016, DeepMind's AlphaGo defeated Lee Sedol, the world champion of the board game Go. Go has more possible board positions than there are atoms in the observable universe, making brute-force search impossible. AlphaGo combined deep neural networks with reinforcement learning to develop strategic intuition, proving that deep learning could master tasks that rely on intuition and long-term strategy, not just pattern recognition.

4.13 Transformer – 2017

In 2017, researchers at Google published the landmark paper “Attention Is All You Need,” introducing the Transformer architecture described in Chapter 3. By replacing sequential processing with parallel self-attention, Transformers enabled models to be trained on vastly larger datasets, directly enabling the rise of modern Large Language Models.

4.14 Modern Large Language Models

Building on the Transformer architecture, researchers scaled up both model size and training data to create Large Language Models capable of understanding and generating human language with remarkable fluency. These models power modern assistants such as ChatGPT, Gemini, and Claude.

4.15 Generative AI Era

Today's AI landscape is defined by generative systems that can produce text, images, audio, and video. This era combines the ideas covered throughout this book — deep neural networks, attention mechanisms, and large-scale training — with the generative architectures covered in Chapter 3 and Chapter 5, such as GANs and diffusion models.

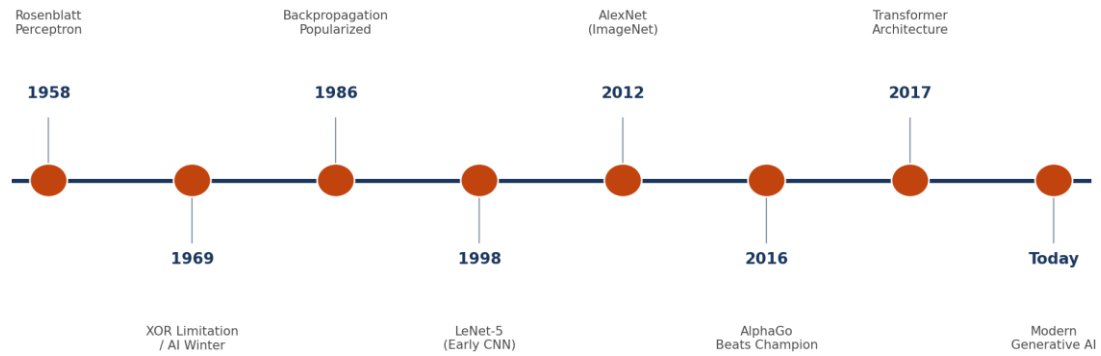


Figure 4.1. Timeline of Major Milestones in Deep Learning History

Table 4.2. Summary Timeline of Deep Learning History

Year	Milestone	Why It Matters
1958	Rosenblatt Perceptron	First working artificial neuron model.
1969	XOR Limitation	Proved single layers cannot solve non-linear problems; began the first AI Winter.
1986	Backpropagation Popularized	Enabled practical training of multi-layer deep networks.
1998	LeNet-5	First practical CNN, used commercially for digit recognition.
2012	AlexNet (ImageNet)	GPUs and deep learning proved vastly superior to traditional machine learning.
2016	AlphaGo	Proved deep networks can master human strategy and intuition.
2017	Transformer	Created the foundation for modern generative AI and large language models.
Today	Generative AI Era	Deep learning powers text, image, audio, and video generation at scale.

Chapter Summary

- The Perceptron (1958) was the first working artificial neuron but could not solve non-linear problems.
- The XOR limitation led to the first AI Winter, a period of reduced funding and progress.
- Backpropagation (1986) made training multi-layer networks practical, ending the first AI Winter.

- AlexNet's 2012 ImageNet win, powered by GPUs and ReLU, marked the start of the modern deep learning era.
- AlphaGo (2016) showed deep learning could master complex strategic intuition.
- The Transformer (2017) enabled parallel training at massive scale, powering today's large language models.

Key Terms

AI Winter • AlexNet • AlphaGo • Backpropagation • GPU Acceleration • ImageNet • LeNet-5 • Perceptron • ReLU • Transformer • XOR Problem

Review Questions

1. Why could the original Perceptron not solve the XOR problem?
2. What caused the first AI Winter, and what ended it?
3. Explain the two key innovations that made AlexNet successful in 2012.
4. Why were GPUs important for the growth of deep learning?
5. What did AlphaGo prove about the capabilities of deep reinforcement learning?
6. Why was the 2017 Transformer paper considered a turning point for AI?

CHAPTER 5

Practical Applications and Real-World Use Cases

The previous chapters explained how neural network architectures work. This chapter shows how those architectures are used to solve real problems, organized into three parts: computer vision, sequential and speech processing, and autonomous systems with generative media.

Part A: Computer Vision

5.1 Image Classification

Image classification is the task of assigning a single label to an entire image, such as recognizing that a photo shows a cat, a car, or a mountain. This is one of the earliest and most common applications of CNNs, discussed in Chapter 1.

5.2 Super-Resolution

Super-resolution is the task of converting a low-resolution image into a sharper, high-resolution version. Traditional methods, such as simple interpolation, only stretch existing pixels, producing blurry results. Deep learning models — often CNNs, Autoencoders, or GANs — instead predict realistic missing detail based on patterns learned from millions of high-resolution training images.

Low-resolution image → Deep learning model (CNN / Autoencoder / GAN) → High-resolution image

- **Medical imaging:** enhancing low-dose MRI or X-ray scans so radiologists can spot small abnormalities without exposing patients to extra radiation.
- **Security and forensics:** restoring license plates or facial detail from grainy surveillance footage.
- **Legacy media preservation:** converting old, low-resolution film and photographs into sharp, modern-quality visuals.

5.3 Image Colorization

Image colorization uses a deep CNN to add realistic color to grayscale photographs and footage. The network studies the shapes, textures, and context in the grayscale image — such as foliage, skin, or sky — and predicts appropriate color values based on patterns it learned from color training data. This is valuable for restoring historical footage and archival photography for modern audiences.

5.4 Object Detection

Object detection goes further than classification: instead of only naming what is in an image, it also identifies where each object is located and draws a bounding box around it. This is essential for systems that must react to the position of objects in real time, such as self-driving cars.

5.5 YOLO

YOLO stands for “You Only Look Once,” a family of object detection architectures that processes an entire image in a single pass through a CNN, rather than scanning the image many times. This single-pass design makes YOLO extremely fast, which is why it is popular for real-time applications.

- **Object detection:** the overall task of finding and labeling objects within an image or video frame.
- **Bounding boxes:** rectangles drawn around each detected object, marking its location and size.
- **Class labels:** the category assigned to each detected object, such as “car” or “person.”
- **Confidence scores:** a number indicating how certain the model is that a detected object belongs to its predicted class.

One CNN pass predicts boxes and class labels for every object at once

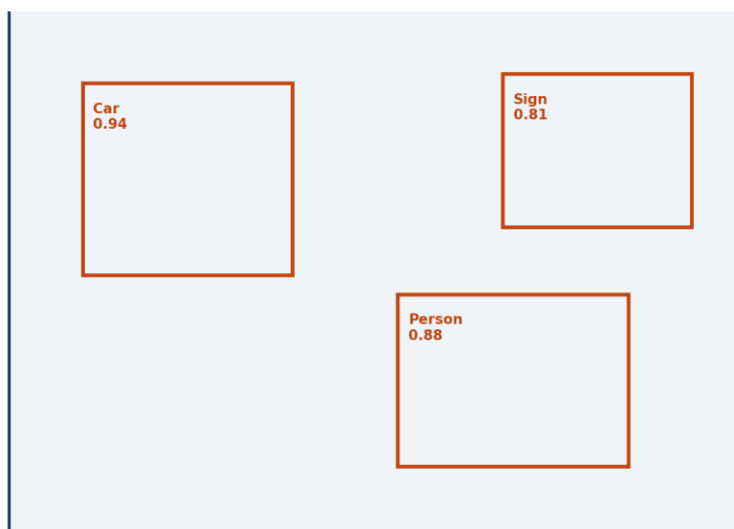


Figure 5.1. YOLO-Style Object Detection with Bounding Boxes

5.6 Medical Imaging

Beyond super-resolution, CNNs and related architectures are widely used to detect tumors, fractures, and other abnormalities directly in CT, MRI, and X-ray scans, helping clinicians target treatment more precisely.

5.7 Security and Surveillance

Object detection and face recognition systems are used in security contexts to identify people or flag unusual activity in camera footage in real time.

5.8 Manufacturing Quality Control

On factory assembly lines, CNN-based object detection systems scan products at high speed to catch defects such as missing components, structural cracks, or misaligned packaging, far faster and more consistently than manual visual inspection.

Part B: Sequential and Speech Processing

5.9 Machine Translation

Machine translation converts text from one language to another. Because languages differ in grammar, word order, and idiom, word-for-word translation fails. Modern systems use Encoder-Decoder Transformer architectures: the encoder reads the source sentence using self-attention to understand its full context, and the decoder generates the translated sentence word by word, staying attentive to the relevant parts of the source text.

5.10 Text Summarization

Text summarization condenses long documents, such as legal contracts or research papers, into short, accurate summaries. This is typically performed by the same class of Transformer-based encoder-decoder models used for translation.

5.11 Speech Recognition

Speech recognition, also called speech-to-text (STT), converts spoken audio into written text. Deep learning models first convert the raw audio waveform into a spectrogram — a visual representation of audio frequency over time — and then use deep neural networks, often combining CNNs and Transformers, to transcribe the speech into text.



Figure 5.2. *The Speech Recognition Pipeline*

5.12 Whisper

OpenAI's Whisper is a widely used speech recognition model. It is important to describe Whisper correctly: Whisper is a speech-to-text model, not a voice synthesis model. It converts spoken audio into text using deep neural networks trained to process spectrograms, as described in Section 5.11.

IMPORTANT

Whisper performs speech recognition (speech-to-text), not text-to-speech synthesis. Confusing the two is a common mistake.

5.13 Text-to-Speech

Text-to-Speech (TTS), also called speech synthesis, is the reverse task of speech recognition: converting written text into natural-sounding spoken audio. Generative models convert text into acoustic representations and then use a neural vocoder to synthesize realistic speech, including natural pauses and intonation.

Table 5.1. Speech Recognition vs Speech Synthesis

Task	Direction	Also Called	Example Model
Speech Recognition	Audio → Text	Speech-to-Text (STT)	Whisper
Speech Synthesis	Text → Audio	Text-to-Speech (TTS)	Tacotron, WaveNet

5.14 ECG Signal Processing

Electrocardiogram (ECG) signals record the heart's electrical activity over time and contain high-frequency noise from muscle movement and electrical interference. Missing a small but important spike or irregular rhythm in this signal can be life-threatening, which makes accurate automated processing valuable in clinical settings.

5.15 ECG Denoising

Denoising autoencoders, introduced in Chapter 2, are commonly used to clean raw ECG voltage readings, filtering out muscle noise and baseline drift so the true cardiac signal is easier to analyze.

5.16 Heartbeat and Arrhythmia Analysis

Once a signal is cleaned, LSTM networks analyze the sequence of heartbeats over time to detect hidden patterns such as arrhythmias or atrial fibrillation, supporting both wearable health monitors and clinical monitoring systems in intensive care units.

5.17 CNN + LSTM / Transformer for Signal Processing

Many real-world signal processing systems combine architectures: a CNN extracts local features from short windows of the signal, while an LSTM or Transformer tracks how those features change over longer stretches of time. This hybrid approach benefits from both the feature-extraction strength of CNNs and the temporal-modeling strength of sequence architectures.



Figure 5.3. A Hybrid CNN + LSTM Pipeline for ECG Signal Analysis

Part C: Autonomous Systems and Generative Media

5.18 Self-Driving Systems

A self-driving vehicle must process multiple camera feeds along with radar and LiDAR sensor streams instantly, recognize the road and other agents, predict how they will move, and execute safe driving decisions within milliseconds. This requires combining multiple deep learning components into a single pipeline.



Figure 5.4. A Simplified Self-Driving Perception and Control Pipeline

- **Perception layer (CNNs):** detects lane lines, traffic lights, vehicles, and pedestrians in real time.
- **Prediction layer (Transformers / RNNs):** predicts where moving objects, such as a pedestrian, will be a few seconds into the future.

- **Control layer (Deep Reinforcement Learning):** calculates the optimal steering, acceleration, and braking based on safety-oriented reward signals.

5.19 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines deep neural networks with trial-and-error learning. An agent takes actions in an environment and receives rewards or penalties based on the outcome, gradually learning a strategy, or policy, that maximizes long-term reward.

5.20 Game-Playing AI

In games such as Go, chess, or complex real-time strategy games, the number of possible move sequences is far too large for brute-force search. DRL agents instead play millions of games, often against copies of themselves, gradually developing strategic instincts through self-play.

5.21 AlphaGo and AlphaZero

AlphaGo, introduced in Chapter 4, defeated the world Go champion using deep reinforcement learning. Its successor, AlphaZero, generalized the same self-play approach to master chess and shogi as well, learning purely from the rules of each game rather than from human game records. Deep reinforcement learning is also applied outside of games, such as optimizing energy usage in data centers and adjusting trading strategies in financial markets.

5.22 Diffusion Models

A diffusion model generates images by starting from pure random noise and gradually removing a small amount of noise at each step, repeating this process many times until a clear, detailed image emerges. Each denoising step is guided by information about what the final image should contain, such as a text prompt.



Guided by a text prompt at every denoising step

Figure 5.5. *How a Diffusion Model Turns Noise into an Image*

5.23 Text-to-Image Generation

Text-to-image systems, such as Stable Diffusion and similar tools, combine a text-understanding component with a diffusion model, using the text prompt to guide each denoising step so the final image matches the description provided by the user.

5.24 Multimodal Generative AI

Multimodal Transformers, such as GPT-4o and Gemini, are trained on text, images, and audio together within a shared representation space. This allows a single model to accept an image and respond with text, or accept text and produce an image, moving beyond the earlier generation of single-mode models that could only handle one type of data.

5.25 Code Generation

Transformer-based models trained on large amounts of source code can write, explain, and debug software, functioning as AI coding assistants directly inside developer tools.

5.26 Creative Applications

Generative models are also used to create backing tracks, story scripts, concept art, and synthetic voice tracks for multimedia production, extending deep learning's reach into creative industries.

Table 5.2. Application Comparison Matrix

Application	Primary Architecture	Why It Is Used
Image classification	CNN	Efficiently extracts visual features from images
Super-resolution	Autoencoder / GAN	Predicts realistic missing detail
Object detection (YOLO)	CNN	Fast, single-pass localization and classification
Machine translation	Transformer	Understands full-sentence context via self-attention
Speech recognition	CNN + Transformer	Converts spectrograms into accurate text
ECG analysis	CNN + LSTM	Combines feature extraction with temporal pattern tracking
Self-driving perception	CNN + Transformer/RNN + DRL	Combines perception, prediction, and control
Game-playing AI	Deep Reinforcement Learning	Learns strategy through self-play and reward
Text-to-image generation	Diffusion Model	Iteratively refines noise into a detailed image
Chatbots / LLMs	Transformer	Parallel processing enables massive-scale language training

Chapter Summary

- CNNs power core vision tasks: classification, super-resolution, colorization, and object detection.
- YOLO performs fast, single-pass object detection using bounding boxes and confidence scores.
- Whisper is a speech-to-text model, not a text-to-speech model.
- ECG signal processing typically combines denoising autoencoders, CNNs, and LSTMs.
- Self-driving systems combine perception (CNN), prediction (Transformer/RNN), and control (DRL).
- Diffusion models generate images by iteratively removing noise, guided by a text prompt.
- Multimodal Transformers process text, images, and audio within a single unified model.

Key Terms

Bounding Box • Diffusion Model • Deep Reinforcement Learning • Multimodal AI • Object Detection • Spectrogram • Super-Resolution • Text-to-Speech • Speech Recognition • YOLO

Review Questions

1. Explain why traditional interpolation produces blurry results in super-resolution, and how deep learning improves on it.
2. What do bounding boxes and confidence scores represent in object detection?
3. Correct the following statement: “Whisper is a text-to-speech model.”
4. Describe the three layers of a self-driving perception and control pipeline.
5. How does a diffusion model turn random noise into a finished image?
6. What distinguishes a multimodal AI model from earlier unimodal models?
7. Choose one application from this chapter and explain which architecture powers it and why.

CHAPTER 6

Advanced and Frontier Topics

Deep learning research today is no longer focused only on classification or prediction. Modern research aims to make AI larger and more capable (scaling), more efficient to run (efficiency), able to understand many types of data at once (multimodal), and safer and more trustworthy (safety and alignment). This chapter introduces these frontier topics.

6.1 Foundation Models

In the past, a separate small model was typically built for each task — one model for translation, another for text classification. A Foundation Model is instead a single, very large model trained on a massive amount of general data, which can then be adapted to perform hundreds of different tasks.

6.2 Large Language Models

Large Language Models, introduced in Chapter 3, are the most well-known category of foundation model, trained on enormous amounts of text using the Transformer architecture.

6.3 Pretraining

Pretraining is the initial, large-scale training stage in which a foundation model learns general patterns of language, images, or other data from a broad dataset, before being adapted to any specific task.

6.4 Fine-Tuning

Fine-tuning takes a pretrained foundation model and continues training it on a smaller, more specific dataset, adapting its general knowledge to a particular task or domain, such as legal document analysis or medical question answering.

6.5 Scaling Laws

Scaling laws describe a consistent pattern observed in deep learning research: as the amount of training data, the size of the model (its number of parameters), and the amount of computing power used for training all increase together, model performance tends to improve in a predictable way.

6.6 Parameters

Parameters are the weights and biases inside a neural network that are adjusted during training. Modern large language models can contain from billions to over a trillion parameters, which is one reason they require enormous computing resources to train.

6.7 Prompt Engineering

Prompt engineering is the practice of carefully wording an instruction, or prompt, given to a trained model in order to obtain a better or more accurate response, without changing the model's underlying code or weights.

6.8 RLHF

Reinforcement Learning from Human Feedback (RLHF) is a training technique in which human reviewers rank or rate a model's responses, and this feedback is used to further train the model to produce answers that are more helpful, accurate, and safe. RLHF is typically applied after pretraining, as part of fine-tuning a foundation model into a usable assistant.

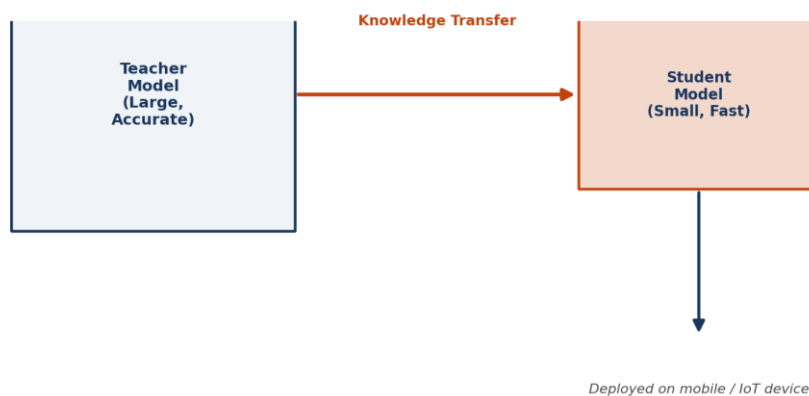


Figure 6.1. Knowledge Distillation: Teacher Model to Student Model

The same teacher-student idea used in RLHF-style fine-tuning also appears in model compression, covered in Section 6.15.

6.9 Multimodal AI

Multimodal AI refers to models that process more than one type of data — such as text, images, audio, video, and sensor data — within a single unified system, discussed in Chapter 5 in the context of generative applications.

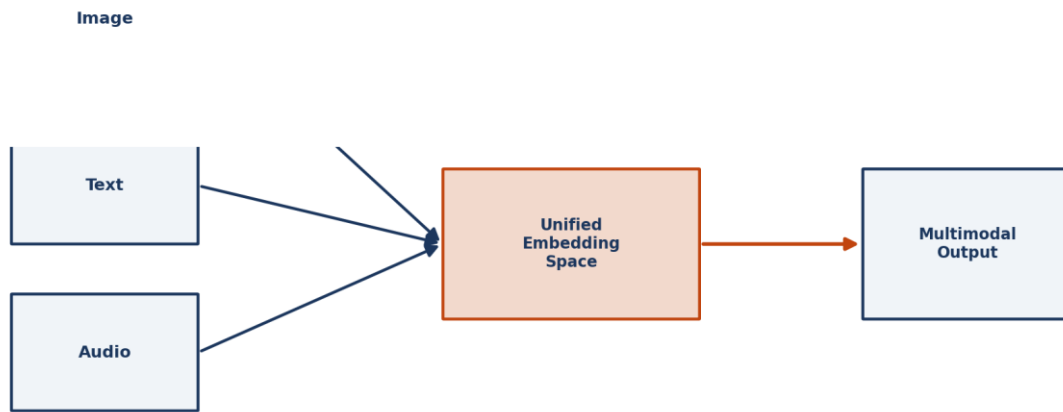


Figure 6.2. A Multimodal AI System Combining Image, Text, and Audio

6.10 Unified Representations / Embeddings

To process multiple data types together, a multimodal model converts each type of input — an image, a sentence, a sound clip — into a shared numerical format called an embedding, introduced in Chapter 2. Placing all data types into this shared embedding space allows the model to relate an image of a dog to the written word “dog” or the sound of a bark.

6.11 Edge AI

Edge AI refers to running AI models directly on local devices — such as smartphones, smartwatches, or IoT sensors — rather than sending data to a remote server for processing. This reduces latency, saves bandwidth, and can improve privacy, since the raw data does not need to leave the device.

6.12 Why Large Models Are Difficult to Run on Small Devices

Modern large models require significant memory, processing power, and electricity to run. Small devices, such as phones and wearables, have far less memory and processing capacity, making it difficult to run a full-sized model directly on-device without special optimization techniques.

6.13 Quantization

Quantization reduces a model's memory footprint by lowering the numerical precision used to store its weights, converting them from 32-bit floating point numbers (FP32) down to 8-bit integers (INT8) or even 4-bit integers (INT4). This can shrink a model's size by more than 75 percent while keeping accuracy close to the original.

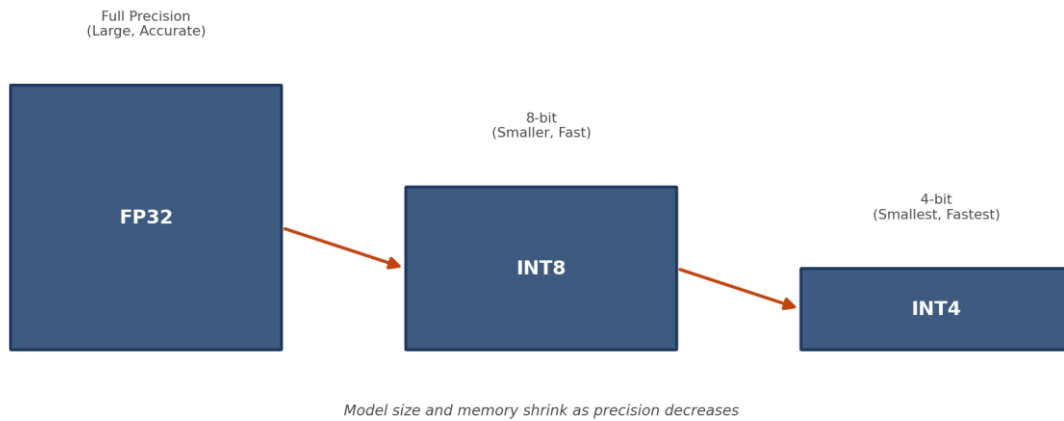


Figure 6.3. Quantization: Reducing Precision from FP32 to INT8 to INT4

There is a trade-off involved: lower precision generally reduces model size and increases speed, but can slightly reduce accuracy if pushed too far. Choosing the right level of quantization balances size, speed, memory usage, and accuracy for the target device.

REAL-WORLD EXAMPLE

Quantization is what makes it possible to run a capable AI model directly on a mobile phone, shrinking it enough to fit in limited memory while keeping performance largely intact.

6.14 Pruning

Pruning removes connections or entire neurons from a trained network that contribute little to its output, reducing the model's size and computational cost while aiming to preserve accuracy.

6.15 Knowledge Distillation

Knowledge distillation trains a smaller “student” model to mimic the behavior of a larger, more capable “teacher” model. The student model learns to reproduce the teacher's outputs, allowing it to achieve much of the teacher's capability while requiring far less memory and computation, making it suitable for deployment on mobile or IoT devices.

6.16 Model Efficiency

Quantization, pruning, and knowledge distillation are all examples of model efficiency techniques. These methods are often combined to fit powerful models onto resource-constrained hardware without a large loss in accuracy.

6.17 Interpretability

Interpretability is the degree to which a human can understand why a model produced a particular output. Deep neural networks are often called “black boxes” because their internal decision-making process is not directly visible or easy to explain.

6.18 Explainable AI

Explainable AI (XAI) is a set of techniques designed to make model decisions more understandable to humans, which is especially important in high-stakes domains such as medicine and autonomous vehicles, where an unexplained wrong decision can be dangerous.

6.19 Mechanistic Interpretability

Mechanistic interpretability is a research area focused on analyzing exactly which internal components — such as specific neurons or attention patterns — a network uses to reach a particular decision, aiming for a much deeper level of understanding than typical explainable AI methods provide.

6.20 AI Hallucination

AI hallucination refers to a model confidently producing information that is incorrect or fabricated, without any indication of uncertainty. This is a well-known limitation of current language models and an active area of research.

6.21 Hallucination Reduction

Researchers use several strategies to reduce hallucination, including training models to express uncertainty, grounding model responses in retrieved factual sources, and using human feedback (such as RLHF) to discourage confidently incorrect answers.

6.22 AI Safety

AI safety is the broad field concerned with ensuring that AI systems behave reliably, avoid causing harm, and remain under meaningful human control as they become more capable.

6.23 AI Alignment

AI alignment is the specific goal of ensuring an AI system's behavior matches human values and intentions — in other words, making sure the system tries to do what humans actually want it to do, and avoids taking harmful actions even when not explicitly told not to.

6.24 Responsible AI

Responsible AI refers to the broader practice of developing and deploying AI systems with attention to fairness, transparency, privacy, and accountability, alongside technical safety and alignment concerns.

6.25 Future Direction of Neural Networks

The trends covered in this chapter — larger foundation models, greater efficiency for everyday devices, richer multimodal understanding, and stronger safety guarantees — are likely to continue

shaping the field. A solid understanding of the core architectures covered in Chapters 1 through 3 remains the foundation for understanding whatever comes next.

Chapter Summary

- Foundation models are large, general-purpose models later adapted through fine-tuning.
- Scaling laws describe how performance improves as data, parameters, and compute increase together.
- RLHF uses human feedback to make model behavior more helpful and safe.
- Multimodal AI combines text, image, audio, and other data types in a shared embedding space.
- Quantization, pruning, and knowledge distillation make large models efficient enough for edge devices.
- Interpretability and explainable AI aim to make model decisions understandable to humans.
- AI safety and alignment focus on ensuring AI systems behave reliably and match human intentions.

Key Terms

AI Alignment • AI Safety • Edge AI • Explainable AI • Fine-Tuning • Foundation Model • Hallucination • Knowledge Distillation • Mechanistic Interpretability • Multimodal AI • Parameters • Pretraining • Pruning • Quantization • RLHF • Scaling Laws

Review Questions

1. What is the difference between pretraining and fine-tuning?
2. Explain scaling laws in your own words.
3. Describe how RLHF changes a model's behavior.
4. Why is quantization necessary for running AI models on mobile devices?
5. Compare pruning and knowledge distillation as model efficiency techniques.
6. What is the difference between AI safety and AI alignment?
7. Why are deep neural networks often described as “black boxes”?

Book Summary

This book has covered the full landscape of neural network architectures, from the basic artificial neuron to the Transformer models behind today's most capable AI systems. Chapter 1 established the foundations: neurons, weights, activation functions, forward propagation, backpropagation, and Convolutional Neural Networks for visual data. Chapter 2 introduced architectures for sequential data — RNNs, LSTMs, and Autoencoders — and explained how they handle order and compression. Chapter 3 covered generative and modern architectures, including GANs and the Transformer's self-attention mechanism.

Chapter 4 placed these ideas in historical context, tracing the path from the 1958 Perceptron through the first AI Winter, the 1986 backpropagation breakthrough, the 2012 AlexNet moment, and the 2017 Transformer paper that launched the modern generative AI era. Chapter 5 connected every architecture to real-world applications across computer vision, speech and language processing, and autonomous and generative systems. Chapter 6 closed the book with a look at the frontier of the field: foundation models, efficiency techniques for running AI on everyday devices, multimodal systems, and the ongoing work of AI safety and alignment.

The architectures in this book are the building blocks of nearly every modern AI system. A reader who understands how a neuron computes, how a network learns through backpropagation, how CNNs process images, how RNNs and LSTMs handle sequences, and how Transformers use self-attention has the foundation needed to keep learning as the field continues to evolve.

Architecture Comparison Matrix

Table A.1. Comparing the Major Neural Network Architectures Covered in This Book

Architecture	Main Purpose	Input Type	Memory	Strength	Weakness	Common Applications
Perceptron	Simple binary decisions	Numeric	None	Simple, fast to compute	Cannot solve non-linear problems (XOR)	Historical / basic logic gates
ANN	General pattern learning	Numeric / tabular	None	Flexible, general-purpose	Inefficient for images or sequences	House price prediction
CNN	Visual feature extraction	Images / video	None	Efficient for spatial patterns	No memory of sequence or order	Image classification, object detection
RNN	Sequence modeling	Sequential data	Short-term	Handles order in short sequences	Vanishing gradient on long sequences	Simple next-word prediction
LSTM	Long sequence modeling	Sequential data	Short and long-term	Handles long sequences well	Slower to train than simple RNN	ECG analysis, text prediction
Autoencoder	Compression and reconstruction	Any data type	None (not sequential)	Learns compact representations	Reconstruction may lose fine detail	Denoising, anomaly detection
GAN	Realistic data generation	Any data type	None	Produces highly realistic output	Training can be unstable	Synthetic image generation
Transformer	Parallel sequence understanding	Sequential / multimodal	Full-context attention	Captures long-range relationships, trains in parallel	Requires large data and compute	Machine translation, LLMs, chatbots
Diffusion Model	Iterative generation from noise	Images / audio / video	None	High-quality, controllable generation	Slow, multi-step generation process	Text-to-image generation

Application Matrix

Table B.1. Applications, Architectures, and Examples

Application	Architecture	Why It Is Used	Example
House price prediction	ANN	Learns numeric relationships from tabular data	Predicting price from size and location
Cat vs dog classification	CNN	Extracts visual features efficiently	Photo app labeling pet photos
Next-word prediction	RNN	Uses short-term memory of prior words	Keyboard word suggestions
ECG sequence analysis	LSTM	Tracks long patterns in a continuous signal	Detecting irregular heartbeats
Image denoising	Autoencoder	Compresses out noise, reconstructs clean signal	Cleaning up a blurry photo
Synthetic image generation	GAN	Generator and Discriminator compete to produce realism	Generating faces of people who do not exist
Machine translation	Transformer	Self-attention captures full-sentence context	Translating a document between languages
Text-to-image generation	Diffusion Model	Iteratively refines noise guided by a text prompt	Generating art from a written description
General-purpose language understanding	Foundation Model / LLM	Broad pretraining transfers to many tasks	A chatbot answering diverse questions
Running AI on a mobile device	Quantized Model	Reduced precision fits limited device memory	An on-device voice assistant

Glossary

Activation Function: A rule applied to a neuron's weighted sum that decides how strongly its signal should pass forward, introducing non-linearity.

Attention: A mechanism that lets a model focus more strongly on the most relevant parts of its input.

Backpropagation: The algorithm that calculates how much each weight contributed to a network's error, working backward from the output.

Bias: An adjustable number added to a neuron's weighted sum, giving the network extra flexibility.

CNN (Convolutional Neural Network): A neural network architecture designed for visual data, using filters that scan across an image.

Convolution: The operation of sliding a small filter across data and computing a weighted sum at each position.

Deep Learning: A branch of machine learning that uses neural networks with many layers to learn patterns automatically.

Embedding: A compact numerical representation of data, such as a word or an image, that captures meaningful patterns.

Epoch: One complete pass of the training data through a neural network during training.

Feature Map: The output produced after a convolutional filter has scanned across an entire input.

GAN (Generative Adversarial Network): A pair of competing networks — a Generator and a Discriminator — trained together to produce realistic new data.

Gradient: A measure of how much a small change in a weight would change the network's loss.

Gradient Descent: The optimization method that updates weights in the direction that reduces loss, using gradients found by backpropagation.

Hidden Layer: A layer of neurons between the input and output layers that performs intermediate calculations.

Hyperparameter: A setting chosen before training, such as the learning rate, that is not learned from data.

Inference: The process of using a trained model to make a prediction on new data.

LSTM (Long Short-Term Memory): A type of recurrent neural network with gated memory cells designed to handle long sequences.

Loss Function: A function that measures how far a model's prediction is from the correct answer.

Neural Network: A system of connected layers of neurons that learns patterns from data.

Neuron: A single computing unit that combines inputs and produces one output value.

Parameter: A weight or bias inside a neural network that is adjusted during training.

Pooling: A downsampling operation in a CNN that shrinks a feature map while retaining its most important information.

ReLU: An activation function that outputs the input directly if positive, and zero otherwise.

RNN (Recurrent Neural Network): A neural network architecture that uses a loop to process sequential data, carrying a hidden state forward in time.

Self-Attention: The mechanism a Transformer uses to weigh the relationship between every pair of positions in a sequence.

Softmax: An activation function that converts raw scores into a probability distribution across multiple categories.

Transformer: A neural network architecture that processes sequences in parallel using self-attention, underlying most modern large language models.

Weight: A number representing how important a particular input is to a particular neuron's calculation.